

* Checks inputs, tool calls, outputs, errors & costs.

LangChain

An open-source framework that connects ~~to~~ LLM's with the outside world - tools, data, APIs and each other.

Features

Connect $\Rightarrow$ Link LLM's to external tools, APIs & databases.

Orchestrate $\Rightarrow$ Chain multiple steps into workflows.

Build $\Rightarrow$ Create Agents, chatbots & RAG applications
Ora

LangChain Ecosystem

- 1) LangChain $\Rightarrow$ Core framework chains, Prompts, Tools, Memory.
- 2) LangGraph $\Rightarrow$ Graph-based Agent Orchestration Engine.
- 3) LangSmith $\Rightarrow$ Observability, Tracing, Testing & Debugging.
- 4) Integrations $\Rightarrow$ 500+ connector for LLMs, Tools, Databases.

Models - Reasoning Engine

* LangChain provides a unified, - but model capabilities differ from the provider.

OpenAI $\Rightarrow$ GPT 5.5, GPT-4o

Anthropic $\Rightarrow$ Claude 3.7 / 3.5 / 4

Google $\Rightarrow$ Gemini 2.5

OpenSource $\Rightarrow$ Llama, Mistral.

Prompt Template $\Rightarrow$ Steering the model.

* Prompt Template $\Rightarrow$ for simple text i/p to o/p tasks

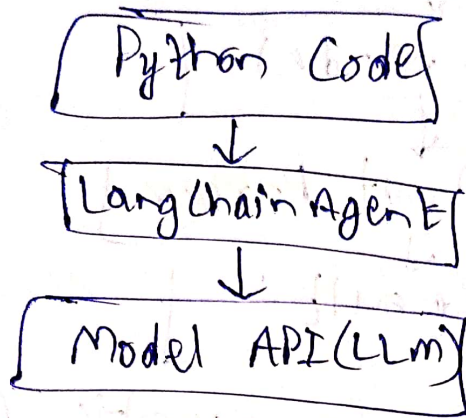
* Chat Prompt Template $\Rightarrow$ for chat models with system, Human, & AI message roles.

* Fewshot Prompts $\Rightarrow$ Include examples in the prompt to improve output quality.

Agent ~~Creation~~ Working

1) User runs the Python Code.

* User never directly communicates with the LLM.



2) LangChain calls Model API

* This calls model API with user messages & available tools.

→ Messages = Conversation so far.

→ Tools = the list of functions model is allowed to call.

Ex: add(), multiply()

3) Model Returns a Tool Call.

* Model cannot execute Python code itself, it returns a tool call request.

4) LangChain interprets the response

* In this step, LangChain understands that LLM did not give a final natural-language answer yet. Instead, it asked for a tool call.

* It checks for,

→ Is there a text answer?

→ Are there tool-calls?

Finally, LangChain extracts three pieces of info:

- * tool-name = "multiply"

- * tool-args = {"a": 15, "b": 8}

- * id = "call-001"

→ LangChain maps tool name to real Python function reference.

- * Internally LangChain maintains a registry of tools. Then it calls the required tool from the registry. From there the selected tool points to the actual Python function.

Model World:
"multiply" → just a string



Python World:
multiply(a,b) → real function

5.) LangChain executes the Python function.

- * Real code runs on your machine.

6.) LangChain sends the Tool result back.

- * LangChain sends the complete message not just the result.

It tells $\Rightarrow$

- * 1) You asked for multiply (15, 8).
- 2) I ran it
- 3) The result is 120.
- 4) Now continue reasoning.

$\Downarrow$
This is because the LLM's are stateless

$\Downarrow$
They do not remember the previous call.

7) Model reasons & request Tool $\rightarrow 2$

$\rightarrow$ Now, model sees the result = 120 & acknowledges it has to be divided with 3.

$\rightarrow$ Each step will build on previous step (multi-step computation).

8) LangChain (interprets) & executes Python Function $\rightarrow 2$ (divide).

Arguments = (previous input, present input).

9) LangChain returns Tool $\rightarrow 2$ result

* This completely returns the cumulative tool result message from the first tool (multiply) to the LLM.

40) Model generates final answer.

Workflow

1st $\rightarrow$ Tool Call (Multiply $\$15, 8$)
2nd $\rightarrow$ Tool Call (Divide $120, 3$)
3rd $\rightarrow$ Final answer = 40.0

} Three Assistant (LLM) responses.

41) Python prints the result.

* Finally, the python code returns the result.

```
agent = create_agent(model=model,  
                      tools=tools)
```

```
result = agent.invoke({"messages": [("user",  
question)]})
```

$\Downarrow$ under this langChain hides these steps.

- $\rightarrow$ Building tool schemas from @tool decorators.
- $\rightarrow$ Formatting JSON messages for the model API.
- $\rightarrow$ Parsing tool call responses from JSON.
- $\rightarrow$ Looking up tools in the internal registry.
- $\rightarrow$ Executing Python functions with parsed args.
- $\rightarrow$ Appending tool results to conversation.
- $\rightarrow$ Deciding if another LLM call is needed.

→ Assembling the final result object.

Key Takeaways

- * LangChain provides a unified interface to work with different LLM providers.
- * Chain = prompt | model | str output parser.
"|" operator chains steps together.
- * LLM = Reasoning only.
The model decides what to call & with what ~~reasoning~~ arguments.
- * Application = Execution.
- * ReAct Loop (Reason → Act → Observe
↓
Repeat).
* Continues until the model needs no more tools.
- * Conversation = State.
- * Framework hides this all ⇒ In one-line agent invoke.